Molarium: Agent-Generated Adaptive Software for Molecular Design

A perspective on persistent molecular state, scientific contracts, and decision provenance in drug discovery

Rafal P. Wiewiora
PsiThera
rafal.wiewiora@psithera.com

W. Patrick Walters
OpenADMET
pat.walters@omsf.io

Woody Sherman
PsiThera
woody.sherman@psithera.com

September 1, 2026

Abstract

Commercial computational drug-discovery software has traditionally bundled three sources of value: the interface, the scientific models, and the specialized expertise required to operate those models and translate their outputs into design decisions. Large language models, coding agents, and tool-using scientific agents are beginning to unbundle this paradigm. Interfaces, analyses, workflow logic, and substantial portions of scientific software can increasingly be generated to meet the needs of a particular scientist, project, and question, while agents can automate sophisticated workflows that once required specialist users.

This shift repositions where competitive advantage is created in computational drug discovery. Faster execution cannot fix an imprecise model, broaden an applicability domain, or enhance the utility of a weak prediction. As the underlying tooling becomes increasingly accessible and fluid, strategic differentiation should shift toward superior and more generalizable models, proprietary experimental data, accumulated discovery intelligence, and the human discernment needed to frame the right questions and evaluate the answers.

Molarium, available at https://molarium.org, provides a case study of this transition. Created during an intensive weekend through collaboration between a drug-discovery scientist and an AI coding agent, this open-source molecular workbench operates primarily in a standard web browser. It integrates local molecular graphics, editing, conformer generation, molecular mechanics and dynamics, machine-learned potentials, and selected protein-structure prediction workflows. Beneath its rapidly adaptable interface, Molarium maintains a versioned molecular state together with user actions, method specifications, provenance, and validation evidence. Decoupling an adaptable interface from an enduring scientific core points toward a broader architecture for generated adaptive software in drug discovery.

Keywords: agentic software development; computational chemistry; molecular design; WebGPU; scientific validation; provenance; discovery intelligence

1 From fixed software products to generated adaptive software

In computational drug discovery, success depends on deploying the right methodology at the right time, supported by appropriate parameters, explicit assumptions, and sound scientific context. Historically, this work has been concentrated among computational specialists and has required substantial operational overhead. Docking depends on protein preparation, protocol selection, constraints, and critical pose evaluation. Molecular dynamics adds system setup, force-field assignment, compute infrastructure, trajectory analysis, and visualization. Free-energy calculations add perturbation design, sampling sufficiency, convergence

assessment, and alignment with experimental conditions. Comparable layers of infrastructure surround cheminformatics, quantum chemistry, property prediction, generative molecular design, and structure prediction.

Commercial scientific software has created considerable value by simplifying and integrating many of these activities. Graphical interfaces reduced technical barriers, workflow engines connected fragmented programs, and software packages encoded operational details needed to apply methods consistently. The same integration coupled scientific questions to relatively fixed software environments. Project-specific improvements competed with broader product roadmaps, new analyses often required engineering support, and scientific users frequently adapted their questions to the capabilities and conventions of the available applications.

Large language models and coding agents are beginning to change this relationship. A scientist with deep drug-discovery expertise but limited programming experience can increasingly describe an application, analysis, visualization, or workflow in natural language and have software constructed around the scientific need. Tool-using agents can operate established methods, link calculations, handle routine failures, and assemble results into forms that support decisions. Earlier systems such as ChemCrow and ChemGraph established proof-of-concept for language models coordinating chemistry tools and computational workflows [1–4]. Their practical scope was limited compared with current coding agents, but they helped establish the feasibility of coupling language models to executable scientific tools.

The consequence is a shift in how drug hunters can interact with computational science. Scientists can increasingly shape software around the question, the stage of the project, the data available at that moment, and their preferred way of reasoning about the problem. A medicinal chemist optimizing a macrocycle may need a different interface and set of calculations than a structural biologist interrogating a cryptic pocket or a program team deciding which compound to advance. The interface can serve as a temporary, adaptive view over a more durable scientific substrate.

Molarium's development directly reflects this paradigm. It began as a need for a more effective environment for viewing and altering molecular structures and expanded through active use. Scientific challenges drove software changes: an unrealistic geometry led to refined editing semantics; questionable energies triggered explicit numerical reference checks; ambiguity about the execution backend led to clearer identification of the active engine; and unsupported methods were converted into explicit boundaries rather than unstated assumptions. The tool evolved alongside the scientific investigation. Appendix A will provide a more detailed account of Molarium's development process, software architecture, supported computational pathways, validation framework, and usage.

2 Molarium and the value of a persistent molecular state

Molarium is an MIT-licensed, local-first molecular viewer, builder, and simulation workbench. Most supported calculations execute on the user's device using WebAssembly or WebGPU, allowing molecular graphics, editing, cheminformatics, conformer exploration, molecular mechanics, molecular dynamics, machine-learned potentials, and selected protein-structure prediction workflows to coexist in a standard browser environment. The browser reduces installation and orchestration overhead and, for supported calculations, can avoid remote job submission, cloud-compute charges, and application licensing while keeping computation inside the interactive design loop. WebAssembly provides a route for mature compiled scientific libraries to run locally, while WebGPU exposes modern accelerators through a portable browser API [5, 6].

The architectural choice that matters most is the preservation of a common scientific state. Molarium maintains explicit topology, coordinates, protonation state, chemical edits, preparation history, trajectories, model identity, and provenance as properties of the molecular session rather than allowing each view or calculation to construct its own interpretation of the molecule. The interface can therefore change rapidly

without redefining the scientific object beneath it.

Three task-specific views illustrate this separation. **View** emphasizes the molecular system, including protein, ligand, waters, ions, contacts, and selected residues. **Design** exposes chemical and geometrical interventions. **Simulate** emphasizes preparation, trajectories, and analysis. The controls differ because the scientific questions differ; topology, coordinates, method identity, and history remain attached to the same evolving molecular state. Detailed examples of these interactions, including three-dimensional molecular editing, local relaxation, simulation, trajectory-frame selection, and subsequent redesign, will be documented in Appendix A.

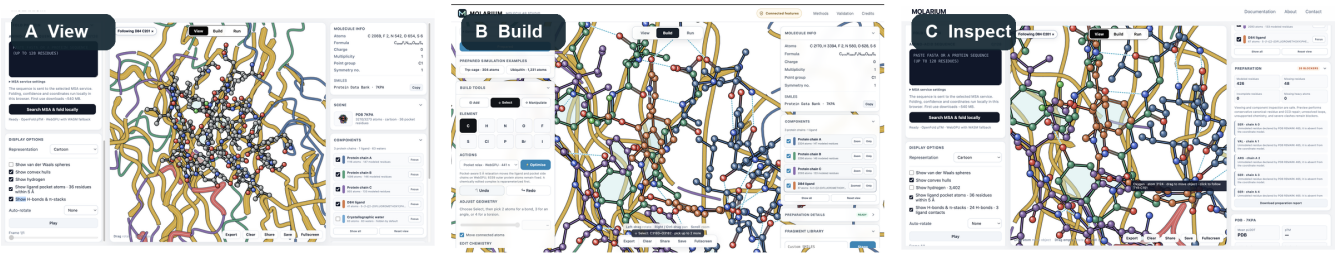

Figure 1: Molarium as an adaptive view over a persistent scientific state. Different views emphasize different design questions while the underlying molecular state, topology, coordinates, history, provenance, and computational context remain connected.

This separation becomes important when interfaces are inexpensive to modify. If every generated view can redefine a molecule, select a different charge model, change protonation, alter preparation, or reinterpret a trajectory, flexibility becomes a source of scientific ambiguity. Generated adaptive software therefore requires a stronger boundary between presentation and scientific state than a conventional fixed interface typically provides.

Molarium extends this principle by preserving the sequence of molecular states and user actions that produced the current session. Edits, selections, manipulations, calculations, and other interactions form an event history rather than leaving only the final coordinates. The resulting architecture is closer to version control than to a traditional molecular project file. Each molecular state can be associated with the actions that produced it, the calculation applied to it, and the evidence available at that point in the design process.

The "Git for molecules" analogy is useful, although a drug-design history is richer than source code. A design trajectory includes structural changes together with evolving hypotheses, model outputs, uncertainty, and experimental evidence. In a discovery setting, such a record could capture which analogs were considered, which structures or properties influenced a design, which alternatives were rejected, and what was learned when a compound was synthesized and tested. This creates a foundation for recording medicinal-chemistry design trajectories rather than storing only the compounds that survived them. Much of the information that explains how a program learned lies in alternatives that were considered and abandoned, yet conventional compound databases usually preserve endpoints.

The browser also provides a useful trust boundary for proprietary work. Molarium's Local Lab mode can restrict network access and verify reviewed application assets while keeping supported calculations on the local device. Because privacy controls depend on deployment configuration, this architecture makes data movement explicit and auditable. Organizations can evaluate and enforce how molecular data are handled rather than treating transfer to remote infrastructure as an invisible property of the software stack.

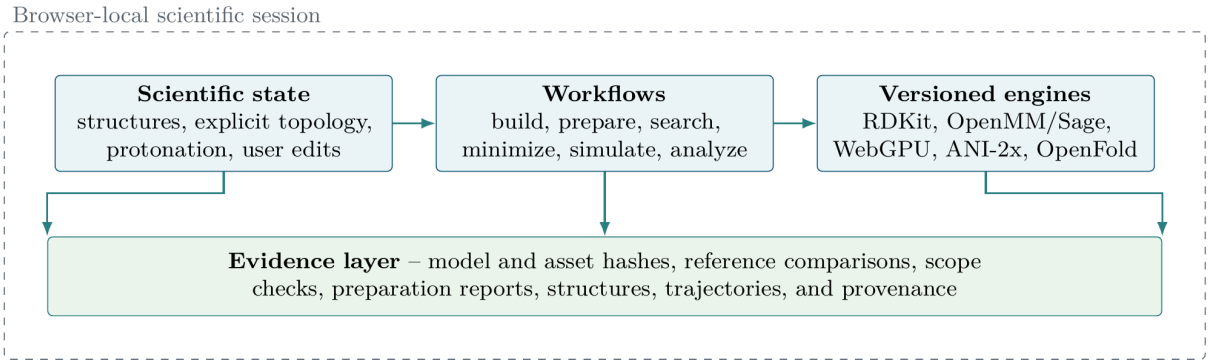

Figure 2: Molarium's local-first architecture. Named computational engines act on an explicit molecular state while provenance and validation evidence remain attached. The same principle provides a foundation for versioned molecular histories in which scientific actions can be replayed and compared without allowing a new interface to redefine the underlying state.

3 Scientific contracts: provenance, validation, and falsifiability

The emergence of cheap, adaptive software introduces a scientific vulnerability. Agentically generated workflows become especially risky when seamless execution obscures the boundaries of the underlying scientific contract. As the time required to build a custom analysis view drops to minutes, that view still needs to expose the molecule, model, parameters, assumptions, and evidence that produced the result. Reducing the friction of software development without those safeguards increases the rate at which scientific ambiguity can be created.

We use the term **scientific contract** for the collection of invariants and evidence that gives a computational output its meaning. A molecular-mechanics result cannot be defined by the generic label "molecular mechanics." Its identity depends on the molecular state, charge assignment, parameterization, energy function, treatment of nonbonded interactions, boundary conditions, constraints, integrator, numerical implementation, and precision. A learned potential depends on its architecture, weights, preprocessing, supported chemistry, and training domain. A protein-structure prediction depends on checkpoint selection, sequence and template information, inference protocol, and other model-specific choices. The surrounding interface may adapt dynamically, but these scientific identities must remain explicit and protected from silent drift.

Molarium preserves distinctions among computational pathways even when they operate in the same workspace. RDKit supports chemical perception and conformer generation; OpenFF Sage defines a molecular-mechanics model; OpenMM Reference provides an independent numerical route for selected validation tests; STORMM-inspired WebGPU code advances homogeneous replicas; ANI-2x provides a machine-learned potential with a defined chemical domain; and OpenFold supports a separate protein-structure prediction pathway [7–12]. These methods can be compared or chained without implying that their energies, assumptions, or domains are interchangeable. Appendix A will specify the implemented pathways and their current boundaries in greater technical detail.

The same principle applies to workflow provenance. Reproducible computational science requires retention of the data, computational process, execution environment, and provenance that produced a result [15, 17]. More recent work formalizes workflow invariants and testable reproducibility criteria rather than treating provenance as passive metadata [16]. Generated adaptive software increases the importance of this requirement because the workflow itself may be created or modified during the scientific interaction rather than existing as a stable artifact defined in advance.

This leads to a stronger requirement: generation of a scientific workflow should be accompanied by generation of the conditions under which its claims could be falsified. If an agent implements a conformer-

search strategy, the output should include the benchmark that tests whether conformational coverage improves, the independent reference used for comparison, the quantitative tolerance that defines agreement, the applicability limits of the test, and explicit failure conditions. If an agent adds a numerical kernel, it should generate the corresponding independent comparison and acceptance criteria. These are better understood as **scientific contract tests** than as ordinary software unit tests because they define what scientific claim the implementation can support.

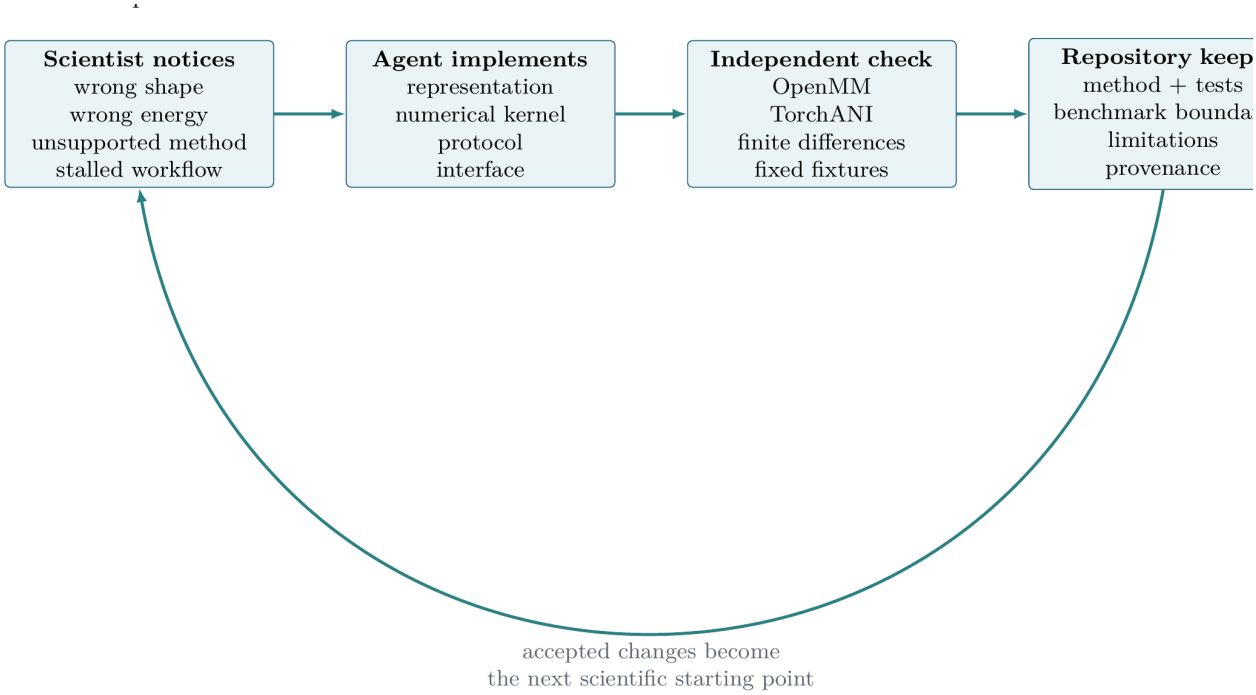

Figure 3: The scientist–agent build loop. Scientific needs and objections are translated into software changes, while each new capability is paired with claim-defining tests, reference evidence, applicability limits, and provenance. The ability to generate a workflow and the ability to specify how its claim could fail are treated as parts of the same scientific result.

Molarium's development repeatedly followed this pattern. A chemically implausible geometry exposed a problem in representation and editing semantics. A questionable energy prompted comparison with an independent implementation. A discrepancy between CPU and GPU execution revealed both a unit error and an incorrect assumption about which backend was running. Requesting constrained dynamics led to implementation of SHAKE/RATTLE and comparison against a reference pathway [13, 14]. In each case, the useful scientific result consisted of the feature together with a test that established the boundary of the claim.

Fluent agentic software makes this distinction more consequential. A unit error can produce a smooth trajectory. Separate implementations can agree if they inherit the same flawed inputs. A numerically correct implementation can apply an inappropriate physical model. Successful execution and an attractive visualization therefore occupy the bottom of an evidence hierarchy rather than establishing that a molecular prediction is correct.

When software construction is difficult, the labor of building a computational method can obscure the separate obligation to validate it. As implementation becomes easier, the scientific responsibility becomes harder to ignore. The workflow, its provenance, and the conditions that could falsify its claims should therefore be treated as a single scientific object.

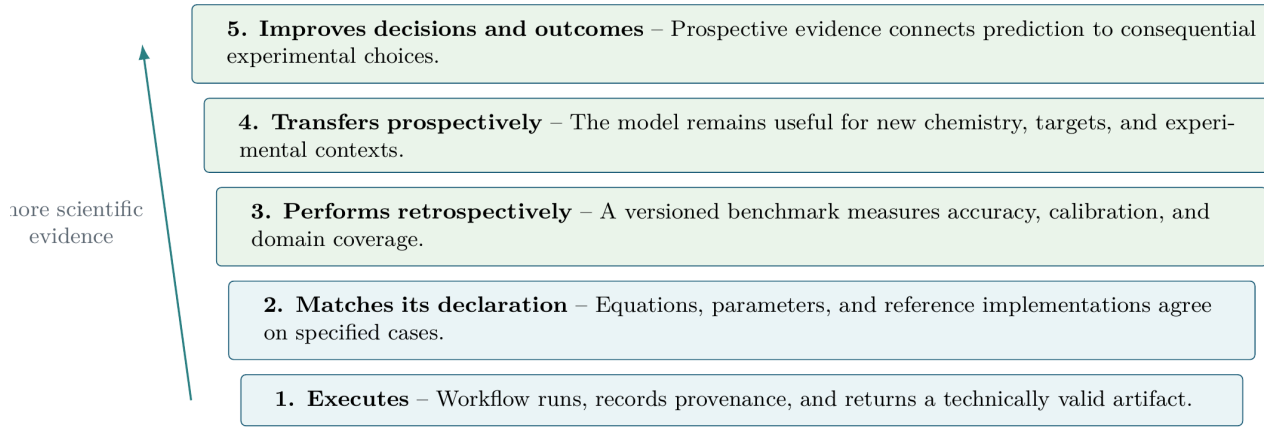

Figure 4: An evidence ladder for generated scientific software. Successful execution establishes less than numerical agreement; numerical agreement establishes less than retrospective predictive performance; retrospective performance establishes less than prospective transfer; and prospective accuracy is distinct from measurable improvement in drug-discovery decisions.

4 Local computation changes the interaction model

A web browser does not replace production molecular simulation software or high-performance computing. Large periodic systems, long trajectories, advanced sampling, specialized force fields, and large-scale parallel studies can require substantially more compute than a personal machine can provide. Molarium addresses a different part of the workflow: calculations that are sufficiently bounded to remain inside the scientist's interactive design session.

The practical benefit is reduced **decision latency**. Consider a medicinal chemist examining a bound ligand and asking whether a proposed three-dimensional edit creates an avoidable steric problem or opens a plausible alternative conformation. In a fragmented workflow, the scientist may export coordinates, request or run a separate calculation, move results back into a viewer, and reconstruct the context in which the question arose. In Molarium, supported editing, local relaxation, conformational exploration, and trajectory inspection can occur in the same session. The value comes from keeping the molecular state, the calculation, and the design question connected while the decision is still active.

Local execution also changes economics for these interactive calculations. The marginal cost of many small calculations approaches the cost of hardware already on the user's desk, without requiring a remote orchestration cycle or a separate software license for each supported operation. This advantage is most relevant in the rapid design loop, where a calculation may be useful because it answers a focused question in minutes rather than because it achieves the maximum possible throughput on a benchmark.

The lower operational barrier increases the importance of choosing calculations deliberately. A medicinal chemist deciding between two substituent orientations may gain value from a constrained local refinement that changes which analog is synthesized next; running hundreds of additional simulations after the decision is already insensitive to the result adds computational activity without increasing scientific leverage. The operational gain matters when it helps the scientist focus on the right question, apply the appropriate method, and prioritize molecules that have a realistic chance to advance the program.

Local execution also redistributes control. The scientist can determine which analysis should exist, how data should be juxtaposed, and which calculation should be available at the point of decision. Molarium illustrates this shift: a scientist did not need to wait for every useful workflow to appear on a conventional product roadmap. Missing interfaces and bounded implementations could be added while the underlying question was still active.

5 What becomes scarce when the tool layer becomes cheap

A concrete medicinal-chemistry example helps separate the layers of value. Suppose a project team is considering a substitution intended to improve a ligand's interaction with a newly identified pocket while preserving permeability. The **tool layer** provides the editor, visualization, coordinate handling, workflow logic, and connection to the relevant calculations. The **model layer** determines whether the predicted pose, conformational preference, interaction energy, or permeability estimate is credible. The **discovery-intelligence layer** supplies the project-specific context: prior SAR, assay conditions, structural interpretation, known liabilities, synthetic constraints, and the reason the team is considering that particular design at that point in the program. Coding agents can make the first layer easier to construct. The value of the second and third layers depends on scientific accuracy and accumulated knowledge rather than interface complexity.

The tool layer includes graphical interfaces, visualization, format conversion, wrappers, workflow logic, reports, job control, data plumbing, and bounded numerical implementation. These capabilities remain important and require maintenance, but their marginal reproduction cost is falling quickly as coding agents improve. A scientist can increasingly generate a project-specific view or connect an existing computational method without waiting for the corresponding feature to appear in a commercial release.

The model layer includes force fields, quantum-mechanical methods, docking and scoring functions, free-energy models, machine-learned potentials, protein-structure predictors, molecular representations, property models, parameters, weights, uncertainty calibration, and the evidence that establishes their useful domain. Easier invocation does not improve predictive quality. A docking model with an inadequate scoring function remains inadequate when an agent invokes it. A systematic force-field error can become more damaging when automated workflows propagate it across thousands of calculations. A machine-learned property model does not gain extrapolative validity because a language model describes its output fluently, and a protein-structure predictor does not become reliable for a particular conformational state because it is available through a better interface.

Differences among models therefore become more consequential as the mechanics of invoking them become less scarce. Drug discovery frequently requires extrapolation into new chemistry, structural states, and biological contexts rather than interpolation within a well-sampled training distribution [18–20]. Faster access can accelerate good models and weak models alike.

The discovery-intelligence layer includes proprietary experimental data, negative results, project history, assay context, structural interpretation, design rationale, internal protocols, expert judgment, and program objectives. Molarium intentionally contains little of this layer because it is a public workbench built primarily from public scientific components. The same architecture could be connected to internal structures, assay data, pharmacokinetics, target biology, medicinal-chemistry reasoning, and program objectives. That project-specific context can determine which computation is relevant and how much weight its output should receive.

Versioned molecular history provides a bridge into discovery intelligence. Traditional compound databases record what was made and measured well, but they rarely preserve what was considered, why one design was preferred, which model or structure influenced the decision, or what evidence was visible at the time. A persistent action history can capture that decision trajectory.

For example, a chemist might propose adding a substituent after inspecting a pocket in a protein–ligand structure. The project record could retain the parent structure, the molecular edit, the structural hypothesis, the conformational or energetic calculations viewed before synthesis, competing designs that were rejected, and the subsequent experimental measurements. If the design later fails because an assumed interaction was absent or a liability dominated the profile, that information remains associated with the decision that created the molecule. Future scientists and agents can then ask which assumptions repeatedly led to productive designs and which repeatedly failed. This is **decision provenance**: preserving the relationship

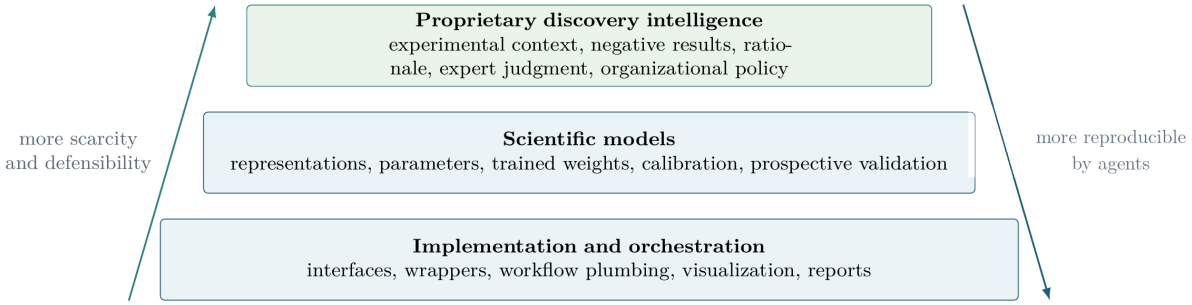

Figure 5: Three interdependent layers of value in generated adaptive molecular design. Implementation and orchestration are becoming increasingly reproducible by coding agents. Scientific models remain differentiated by accuracy, representation, domain coverage, calibration, and validation. Proprietary discovery intelligence combines experimental data with the context and decision history that make those data useful.

among hypotheses, interventions, predictions, decisions, and outcomes rather than storing only final compounds and assay values.

Agentic systems provide a mechanism to operationalize some of this accumulated knowledge through **scientific skills**. Here, a skill means a version-controlled, machine-executable procedure that combines a defined tool or model with an organization's accepted inputs, assumptions, validation criteria, applicability limits, failure handling, and escalation rules. A protein-preparation skill might encode how crystallographic waters, protonation states, alternate conformations, unusual residues, and ambiguous cases are handled. A free-energy skill might define accepted perturbations, convergence criteria, diagnostics, and failure modes. The durable asset is the scientific procedure and its validation, rather than the natural-language prompt used to invoke it.

6 The changing role of scientific expertise

As software becomes easier to construct and operate, scientific judgment becomes more valuable. Traditional computational expertise includes substantial operational knowledge: preparing inputs, locating settings, translating files between programs, recovering failed jobs, and reproducing standard analyses. An increasing fraction of this work can be encoded in software and agents. The broader automation literature has long recognized that reducing routine operation can increase the importance of human expertise when systems encounter ambiguous or abnormal conditions [21].

The highest-value expertise shifts toward selecting the right question, choosing an appropriate level of model fidelity, defining acceptable tradeoffs between cost and accuracy, recognizing implausible results, selecting meaningful controls, and determining which evidence would change a project decision. Scientists closest to the underlying discovery problem—whether medicinal chemists, structural biologists, mechanistic biologists, pharmacologists, or computational scientists—can increasingly interact directly with relevant data and calculations rather than routing every question through a software intermediary.

This shift changes the leverage of computational specialists. Less specialist time needs to be spent constructing repetitive interfaces, transferring files, or manually operating routine workflows. More can be spent on model development, molecular physics, parameterization, sampling, uncertainty, validation, and difficult edge cases where established workflows fail. Once that expertise is made explicit, part of it can be encoded into validated scientific skills that broaden access without implying that the underlying science has become simple.

Molarium's development illustrates this division of labor. The coding agent could inspect a repository, refactor software, implement numerical kernels, add tests, and repeat the loop rapidly. The scientist had to

recognize chemically implausible results, decide whether a reference was genuinely independent, determine whether a discrepancy mattered, choose acceptable approximations, and identify what evidence would change a decision. The agent compressed the distance between recognizing a scientific need and changing the software; scientific judgment determined which changes mattered.

The bottleneck therefore moves toward epistemology: deciding which approximation is acceptable, whether a faster method sacrifices information that matters to the decision, when an independent control is required, whether a reference is truly independent, whether a model is operating outside its validated domain, what uncertainty must remain visible, and which experiment would discriminate between competing hypotheses. Faster implementation increases the value of asking the right question.

7 Limitations and outlook

Molarium is a case study rather than a controlled experiment in software productivity or drug-discovery performance. Its development involved a small number of scientists, one evolving codebase, and a particular coding-agent workflow. The project did not include a randomized comparison with a conventional software-development team, and it cannot establish a universal productivity multiplier or determine what fraction of commercial scientific software can be regenerated reliably.

The open-source repository can nevertheless support a useful community experiment. Contributors can identify computational capabilities described in papers or public specifications, attempt independent agent-assisted reconstructions, test them against appropriate references, and record which parts can and cannot be reproduced reliably. Over time, this could become a living benchmark for the regeneration of molecular-modeling software, with success defined by scientific contract tests rather than by code generation alone.

Molarium's scientific scope is deliberately bounded. Current browser-native molecular mechanics does not reproduce the full capability of mature native simulation environments. General periodic simulation, particle-mesh Ewald electrostatics, barostats, membranes, unrestricted protein parameterization, arbitrary plugins, and other advanced capabilities remain outside the current browser-native scope. ANI-2x remains valid only within its declared chemical domain, and browser-based OpenFold support is narrower than general production protein-structure prediction workflows. Generated wrappers and easier access must preserve, expose, and enforce these applicability limits rather than implying broader scientific validity. Appendix A will catalogue the supported functionality and current limitations in detail.

A second limitation concerns the distinction between implementation fidelity and predictive validity. If a browser implementation reproduces an OpenMM reference energy and force to a specified tolerance, that result establishes that the implementation is executing the declared calculation correctly for the tested case. It does not establish that the shared force field predicts the experimental behavior of a new molecule. Likewise, a conformer workflow can reproduce its intended algorithm and explore more structures without improving recovery of biologically relevant conformations. Implementation validation and model validation answer different questions and require different evidence.

The future suggested by Molarium is one in which the interface between scientist and computation becomes more fluid while the scientific state, models, assumptions, evidence, and decision history become more explicit. Project teams can construct temporary views around specific questions; internal models can coexist with open-source methods and commercial services; and the user experience no longer needs to be owned by the provider of each underlying calculation.

The durable infrastructure in such an environment is concrete. A user should be able to inspect the active molecular state and its history, the exact model and parameter versions used for each calculation, declared applicability limits, validation status, assumptions, provenance, and links between predictions and subsequent decisions. These quantities should remain available regardless of which generated view is open. An adaptive interface can change rapidly because the scientific contract underneath it remains

stable and inspectable.

8 Conclusion

Molarium began as a request for a better environment in which to inspect and manipulate molecules. It developed rapidly into a local-first molecular-design workbench incorporating topology-aware editing, protein preparation, conformer exploration, molecular mechanics, molecular dynamics, learned potentials, protein-structure prediction, browser-native GPU computation, provenance, and reference validation. The speed of that development is technically interesting, but the broader lesson concerns the architecture of scientific software and the location of durable value.

Software can increasingly be generated around the scientific question and the user's specific preferences. The quality of the underlying models, validation, and decision history becomes more consequential as the interface becomes cheaper and more fluid. The scarce assets shift toward trustworthy models, proprietary discovery intelligence, and the human judgment required to know what to ask and what to believe.

Molarium also suggests that molecular-design environments should preserve more than the latest structure. A versioned record of molecular states, user actions, calculations, assumptions, and results can capture the trajectory by which scientific decisions were made. Coupled to explicit scientific contracts and falsification tests, that history can make generated adaptive software more reproducible, extensible, and scientifically accountable.

When software is difficult to construct, implementation effort can hide the distinction between building a method and validating it. When implementation becomes easy, the remaining scientific obligations become harder to ignore. The opportunity is to place adaptable computational capability directly in the hands of scientists who understand the problem while preserving the models, evidence, and decision history that determine whether the answer should be trusted.

The tool layer is becoming cheap and adaptive. Scientific advantage moves toward better models, better discovery intelligence, and better decisions.

AI assistance and availability

GPT-5.6 Sol was used extensively as a coding agent during the development of Molarium under human scientific direction, inspection, testing, and revision. Language-model assistance was also used in restructuring and drafting this manuscript. The authors remain responsible for the scientific claims, validation, source attribution, and final text.

Molarium is available at `https://molarium.org`. Its source code and technical documentation are publicly available at `https://github.com/rafwiewiora/molarium`. Quantitative benchmark claims in a submitted version should be accompanied by a frozen validation ledger recording the relevant commit, fixtures, model and asset hashes, browser and hardware environment, numerical precision, and timing boundaries.

A Technical implementation, architecture, validation, and usage

This appendix will be provided by Rafal P. Wiewiora. It will describe Molarium's development process, software architecture, browser-local execution pathways, supported computational methods, validation and scientific-contract tests, provenance model, design-history representation, and representative usage workflows.

References

[1] Bran, A. M.; Cox, S.; Schilter, O.; Baldassari, C.; White, A. D.; Schwaller, P. Augmenting Large Language Models with Chemistry Tools. *Nature Machine Intelligence* **2024**, *6*, 525–535.

[2] Boiko, D. A.; MacKnight, R.; Kline, B.; Gomes, G. Autonomous Chemical Research with Large Language Models. *Nature* **2023**, *624*, 570–578.

[3] Pham, T. D.; Tanikanti, A.; Keçeli, M. ChemGraph as an Agentic Framework for Computational Chemistry Workflows. *Communications Chemistry* **2026**, *9*, 33.

[4] MacKnight, R.; Boiko, D. A.; Regio, J. E.; Gallegos, L. C.; Neukomm, T. A.; et al. Rethinking Chemical Research in the Age of Large Language Models. *Nature Computational Science* **2025**, *5*, 715–726.

[5] Haas, A.; Rossberg, A.; Schuff, D. L.; et al. Bringing the Web up to Speed with WebAssembly. *Proc. PLDI* **2017**, 185–200.

[6] W3C GPU for the Web Working Group. *WebGPU Specification*, 2025.

[7] Riniker, S.; Landrum, G. A. Better Informed Distance Geometry: Using What We Know to Improve Conformation Generation. *J. Chem. Inf. Model.* **2015**, *55*, 2562–2574.

[8] Boothroyd, S.; Madin, O. C.; Mobley, D. L.; et al. Development and Benchmarking of Open Force Field 2.0.0: The Sage Small-Molecule Force Field. *J. Chem. Theory Comput.* **2023**, *19*, 3251–3275.

[9] Eastman, P.; Galvelis, R.; Peláez, R. P.; et al. OpenMM 8: Molecular Dynamics Simulation with Machine Learning Potentials. *J. Phys. Chem. B* **2024**, *128*, 109–116.

[10] Cerutti, D. S.; Wiewiora, R. P.; Boothroyd, S.; Sherman, W. STORMM: Structure and Topology Replica Molecular Mechanics for Chemical Simulations. *J. Chem. Phys.* **2024**, *161*, 032501.

[11] Devereux, C.; Smith, J. S.; Huddleston, K. K.; et al. Extending the Applicability of the ANI Deep Learning Molecular Potential to Sulfur and Halogens. *J. Chem. Theory Comput.* **2020**, *16*, 4192–4202.

[12] Ahdritz, G.; Bouatta, N.; Floristean, C.; et al. OpenFold: Retraining AlphaFold2 Yields New Insights into Its Learning Mechanisms and Capacity for Generalization. *Nature Methods* **2024**, *21*, 1514–1524.

[13] Ryckaert, J.-P.; Ciccotti, G.; Berendsen, H. J. C. Numerical Integration of the Cartesian Equations of Motion of a System with Constraints: Molecular Dynamics of n-Alkanes. *J. Comput. Phys.* **1977**, *23*, 327–341.

[14] Andersen, H. C. RATTLE: A "Velocity" Version of the SHAKE Algorithm for Molecular Dynamics Calculations. *J. Comput. Phys.* **1983**, *52*, 24–34.

[15] Cohen-Boulakia, S.; Belhajjame, K.; Collin, O.; et al. Scientific Workflows for Computational Reproducibility in the Life Sciences: Status, Challenges and Opportunities. *Future Gener. Comput. Syst.* **2017**, *75*, 284–298.

[16] Pritchard, N. J.; Wicenec, A. Formal Definition and Implementation of Reproducibility Tenets for Computational Workflows. *Future Gener. Comput. Syst.* **2025**, *166*, 107684.

[17] Wilkinson, M. D.; Dumontier, M.; Aalbersberg, I. J.; et al. The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Sci. Data* **2016**, *3*, 160018.

[18] Bender, A.; Cortés-Ciriano, I. Artificial Intelligence in Drug Discovery: What Is Realistic, What Are Illusions? Part 1. *Drug Discov. Today* **2021**, *26*, 511–524.

[19] Bender, A.; Cortés-Ciriano, I. Artificial Intelligence in Drug Discovery: What Is Realistic, What Are Illusions? Part 2. *Drug Discov. Today* **2021**, *26*, 1040–1052.

[20] Wigh, D. S.; Goodman, J. M.; Lapkin, A. A. A Review of Molecular Representation in the Age of Machine Learning. *WIREs Comput. Mol. Sci.* **2022**, *12*, e1603.

[21] Bainbridge, L. Ironies of Automation. *Automatica* **1983**, *19*, 775–779.